

TypeScript im Browser – Herausforderungen und Lösungsansätze

1. Einführung in TypeScript

1.1 Was ist TypeScript?

TypeScript ist eine von **Microsoft entwickelte** typisierte Obermenge von JavaScript. Jeder gültige JavaScript-Code ist auch gültiger TypeScript-Code – TypeScript erweitert JavaScript um ein optionales Typsystem.

- **Open Source** seit 2012
- **Kompiliert zu** plain JavaScript
- **Statische Typprüfung** zur Compile-Zeit (nicht zur Laufzeit)
- Bietet **bessere IDE-Unterstützung** (Autovervollständigung, Refactoring, Fehlererkennung)

1.2 Warum TypeScript?

JavaScript ist **dynamisch typisiert** – Variablen können jeden Typ annehmen:

```
let x = 42;           // number
x = "Hallo";         // plötzlich string -- kein Fehler!
x = [1, 2, 3];       // jetzt array -- immer noch kein Fehler!
```

TypeScript fügt **Typsicherheit** hinzu:

```
let x: number = 42;
x = "Hallo"; // Fehler: Type 'string' is not assignable to type 'number'
```

Die Vorteile im Überblick:

Aspekt	JavaScript	TypeScript
Typisierung	dynamisch, zur Laufzeit	statisch, zur Compile-Zeit
Fehlererkennung	zur Laufzeit	beim Schreiben / Bauen
IDE-Support	begrenzt	Autovervollständigung, Refactoring
Wartbarkeit	schwerer bei großen Projekten	selbst-dokumentierend durch Typen

1.3 Grundlegende Typen

```
// Primitive Typen
let name: string = "GRG";
let alter: number = 17;
let aktiv: boolean = true;

// Arrays
let zahlen: number[] = [1, 2, 3];
let namen: Array<string> = ["Anna", "Ben"];

// Objekttypen (Interfaces)
interface Flug {
  code: string;
  destination: string;
  type: "arrival" | "departure"; // Union Type + Literal Type
  zeit: string;
}

const flug: Flug = {
  code: "OS123",
  destination: "Wien",
  type: "arrival",
  zeit: "14:30",
};
```

1.4 Type Inference (Typinferenz)

TypeScript ist **smart genug**, Typen automatisch abzuleiten. Man muss nicht immer alles explizit typisieren:

```
// TypeScript leitet den Typ automatisch ab
let x = 42;           // TypeScript weiß: x ist number
let name = "GRG";     // TypeScript weiß: name ist string

// Funktion mit Rückgabotyp-Inferenz
function addiere(a: number, b: number) {
    return a + b;      // Rückgabotyp wird automatisch als number erkannt
}

// Aber: Bei komplexeren Funktionen ist explizite Typisierung empfehlenswert
function getRequiredElement<T extends HTMLElement>(id: string): T {
    const element = document.getElementById(id);
    if (!(element instanceof HTMLElement)) {
        throw new Error("Missing HTML element: " + id);
    }
    return element as T;
}

// Verwendung mit Generics -- Typ wird vom Aufruf abgeleitet
const canvas = getRequiredElement<HTMLCanvasElement>("chart-canvas");
const button = getRequiredElement<HTMLButtonElement>("refresh-btn");
```

Faustregel: Typen dort explizit angeben, wo es die Lesbarkeit verbessert (Funktionssignaturen, Interfaces). Lokale Variablen können oft inferiert werden.

1.5 Klassen in TypeScript

TypeScript erweitert JavaScript-Klassen um **Typ-Annotationen**, **Sichtbarkeitsmodifikatoren** und **Zugriffskontrolle**:

```
class ScoreBoard extends LitElement {
    static override properties = {
        score: { type: Number },
        mood: { type: String },
    };

    declare score: number;      // Typannotation für Property
    declare mood: string;

    constructor() {
        super();
        this.score = 4;
        this.mood = "Solider Start";
    }

    private increaseScore(): void {    // private + expliziter Return-Typ
        this.score += 1;
        this.mood = this.score > 7 ? "Starker Lauf" : "Solider Start";
    }

    private resetScore(): void {
        this.score = 0;
        this.mood = "Neu gestartet";
    }
}
```

2. Das Kernproblem: TypeScript läuft nicht im Browser

2.1 Das Problem

Browser verstehen **nur JavaScript**. TypeScript-Code kann nicht direkt ausgeführt werden – er muss zuerst nach JavaScript **transpiliert** werden.

TypeScript-Quellcode (.ts) → Transpiler → JavaScript (.js) → Browser

Anders als bei serverseitigem Code (wo Deno TypeScript nativ ausführt) braucht der Browser einen **zusätzlichen**

Build-Schritt.

2.2 Traditionelle Ansätze und ihre Nachteile

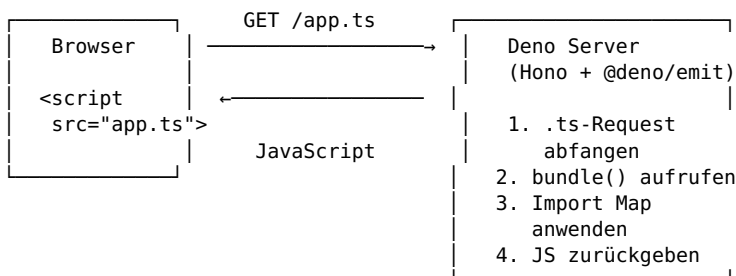
Ansatz	Nachteil
Manuelle Kompilierung (tsc)	Muss nach jeder Änderung manuell ausgeführt werden
Build-Tools (Webpack, Vite, esbuild)	Komplexe Konfiguration, viele Abhängigkeiten
Watch-Modus (tsc --watch)	Nur Transpilation, kein Bundling von npm-Paketen
In-Browser Transpilation (ts-standalone)	Langsam, nicht für Produktion geeignet

2.3 Die Deno-Lösung: Real-time Transpilation

Unser Ansatz nutzt **Deno als Entwicklungsserver**, der TypeScript-Dateien **bei Bedarf transpiliert und bundled** – on the fly, ohne separates Build-Setup.

3. Deno Bundler und Real-time Transpilation

3.1 Architekturüberblick



Das Geniale: Im HTML steht `<script src="app.ts" defer>` – als wäre es eine ganz normale JavaScript-Datei. Der Deno-Server fängt den Request ab, transpiliert, und liefert gültiges JavaScript zurück.

3.2 Der Transpile-Server

Der Server basiert auf **Hono** (einem schlanken Webframework) und **@deno/emit** (Deno's Bundle-API):

```
import { Hono } from "hono";
import { serveStatic } from "hono/deno";
import { bundle } from "@deno/emit";
import denoConfig from "../deno.json" with { type: "json" };

const app = new Hono();
const isDev = true;
const importMap = {
  imports: denoConfig.imports,
};

// Alle Requests auf .ts-Dateien abfangen
app.get("/:path{.+\\.ts$}", async (c) => {
  const filePath = `./src/${c.req.param("path")}`;
  console.log(`Transpiling ${filePath}`);

  try {
    const result = await bundle(filePath, {
      allowRemote: true,
      importMap,
      minify: !isDev,
      type: "module",
    });
    const js = result.code;

    if (!js) throw new Error("Bundling did not produce JavaScript output");

    return c.body(js, 200, {
```

```

        "Content-Type": "application/javascript; charset=utf-8",
        "Cache-Control": isDev ? "no-cache" : "public, max-age=31536000",
    });
} catch (err) {
    const message = err instanceof Error ? err.message : String(err);
    return c.text(`Transpilation Error: ${message}`, 500);
}
});

// Statische Dateien aus ./static
app.use("/*", serveStatic({ root: "./static" }));

Deno.serve(app.fetch);

```

Was passiert hier Schritt für Schritt:

1. **Route-Matcher** `/:path{.+\\.ts$}` – fängt alle Requests ab, die auf `.ts` enden
2. **bundle()** von `@deno/emit` – transpileiert TypeScript und löst alle Imports auf
3. **Import Map** – ersetzt kurze Modulnamen ("`chart-js`") durch echte URLs ("`https://esm.sh/chart.js@4.5.1`")
4. **Content-Type** `application/javascript` – der Browser erhält gültiges JS
5. **Cache-Control** – in Entwicklung: `no-cache`, in Produktion: aggressive Caching

3.3 Die Import Map – Schlüssel zur Browser-Kompatibilität

Die `deno.json` enthält die zentrale Import Map:

```

{
  "imports": {
    "hono": "jsr:@hono/hono@4.12.8",
    "@deno/emit": "jsr:@deno/emit@0.46.0",
    "@std/assert": "jsr:@std/assert@1",
    "canvas-confetti": "https://esm.sh/canvas-confetti@1.9.4?target=es2022",
    "chart-js": "https://esm.sh/chart.js@4.5.1?target=es2022",
    "lit": "https://esm.sh/lit@3.3.2?target=es2022",
    "ms": "https://esm.sh/ms@2.1.3?target=es2022"
  }
}

```

Verschiedene Import-Specifier:

Specifier	Quelle	Verwendung
<code>jsr:@hono/hono</code>	Deno Registry (JSR)	Serverseitige Deno-Pakete
<code>npm:hono</code>	npm Registry	Node-Kompatibilität
<code>https://esm.sh/...</code>	ESM.sh CDN	Browser-kompatible ESM-Pakete
<code>https://deno.land/x/</code>	Deno Land (Legacy)	Ältere Deno-Pakete

3.4 ESM.sh – npm-Pakete für den Browser

ESM.sh ist ein CDN, das **npm-Pakete in das ESM-Format umwandelt**. Das ist entscheidend, weil viele npm-Pakete nur als CommonJS verfügbar sind.

```

// Im TypeScript-Quellcode (wird auf dem Server transpiliert):
import { Chart, registerables } from "chart-js";

// Nach dem Bundling (was im Browser ankommt):
// chart-js wird aufgelöst zu:
// https://esm.sh/chart.js@4.5.1?target=es2022

```

Der `?target=es2022` Parameter stellt sicher, dass modernes JavaScript ausgeliefert wird.

3.5 Praktisches Beispiel: Chart.js-Demo

Im HTML wird TypeScript **direkt referenziert**:

```

<!-- flight-board.html -->
<script src="flight-board.ts" defer></script>

```

Und im TypeScript-Quellcode können wir npm-Pakete verwenden, als wären sie lokal installiert:

```
import { Chart, registerables } from "chart-js";

Chart.register(...registerables);

const chartCanvas = getRequiredElement<HTMLCanvasElement>("chart-canvas");

const chart = new Chart(chartCanvas, {
  type: "bar",
  data: {
    labels: ["Mo", "Di", "Mi", "Do", "Fr", "Sa"],
    datasets: [{
      label: "Besucher pro Tag",
      data: createDataset(),
      borderRadius: 10,
      backgroundColor: ["#1262d6", "#2782e6", "#39a39f",
        "#78c5b8", "#f0b352", "#f07f4f"],
    }],
  },
  options: { responsive: true, maintainAspectRatio: false },
});
```

4. Modulsysteme: require vs. import

4.1 Zwei Welten treffen aufeinander

JavaScript hat **zwei verschiedene Modulsysteme**, die nicht miteinander kompatibel sind:

	CommonJS (CJS)	ECMAScript Modules (ESM)
Syntax	require() / module.exports	import / export
Entstanden	Node.js (2009)	ES6-Standard (2015)
Laden	synchron	asynchron
Gültigkeitsbereich	Funktions-Scope	Modul-Scope (strikter)
Browser	nicht unterstützt	unterstützt (<script type="module">)
Top-level await	nicht möglich	möglich

4.2 CommonJS – Das Node.js-Modulsystem

```
// exportieren (math.js)
module.exports = {
  add: function(a, b) { return a + b; },
  multiply: function(a, b) { return a * b; },
};

// oder: exports.add = function(a, b) { return a + b; };

// importieren
const math = require("./math.js");
math.add(2, 3); // 5

// oder: const { add } = require("./math.js");
```

Merkmale: - require() ist ein **synchroner Funktionsaufruf** – die Datei wird sofort geladen - module.exports ist ein **einzelnes Objekt** - Wird zur **Laufzeit** ausgewertet – dynamische Imports möglich: javascript if (condition) { const mod = require("./optional-modul"); // funktioniert }

4.3 ECMAScript Modules (ESM) – Der Standard

```
// exportieren (essen.ts)
export type EssenEintrag = {
  name: string;
  essen: string;
};

export async function holeEssen(): Promise<void> {
```

```

    const response = await fetch("/essen");
    const daten: EssenEintrag[] = await response.json();
    // ...
}

export function loescheEssen(): void {
    // ...
}

// importieren -- drei Varianten:
import { holeEssen, loescheEssen } from "./essen.ts";
import * as essenFn from "./essen.ts";
import "./essen.ts";
// Named Import
// Namespace Import
// Side-Effect Import

```

Die drei Import-Varianten erklärt:

Variante	Syntax	Wirkung
Named Import	import { foo } from "./mod"	Importiert spezifische Exporte – die bevorzugte Methode
Namespace Import	import * as mod from "./mod"	Erstellt ein Objekt mit allen Exporten (mod.foo, mod.bar)
Side-Effect Import	import "./mod"	Führt das Modul aus, importiert aber keine Werte

4.4 Die Fallstricke bei der gemischten Verwendung

Problem 1: Browser versteht nur ESM

```

<!-- FUNKTIONIERT NICHT -->
<script src="mein-modul.js"></script>

<!-- Korrekt -->
<script type="module" src="mein-modul.js"></script>

```

Ohne type="module" hat das Skript keinen Modul-Scope – import und export sind nicht verfügbar.

Problem 2: Module haben eigenen Scope

```

// script.ts -- ein ES Module
import { holeEssen } from "./essen.ts";

// Diese Funktion ist NICHT global verfügbar!
// Ein onclick="holeEssen()" im HTML würde fehlschlagen!

```

Lösung A – addEventListener statt onclick:

```

// Die saubere Lösung: Event-Listener programmatisch registrieren
document.getElementById("hole-essen")?.addEventListener("click", holeEssen);

```

Lösung B – Explizites Exportieren auf globalThis:

```

import { holeEssen, loescheEssen } from "./essen.ts";

type EssenGlobals = typeof globalThis & {
    holeEssen: typeof holeEssen;
    loescheEssen: typeof loescheEssen;
};

const globals = globalThis as EssenGlobals;
globals.holeEssen = holeEssen;
globals.loescheEssen = loescheEssen;

```

So werden die Funktionen wieder global sichtbar – onclick="holeEssen()" funktioniert dann. Aber **Lösung A (addEventListener) ist vorzuziehen**, weil sie keine globalen Variablen verschmutzt.

Problem 3: CommonJS-Pakete im Browser

Viele npm-Pakete liegen nur als CommonJS vor:

```
// So sieht der Quellcode vieler npm-Pakete aus:
const crypto = require("crypto");
module.exports = { hash: function(data) { ... } };
```

Browser verstehen require nicht! Das ist genau das Problem, das **ESM.sh** löst:

npm-Paket (CommonJS) → ESM.sh CDN → ESM-kompatibles Modul → Browser

Problem 4: `__dirname` und `__filename` existieren nicht in ESM

```
// CommonJS (Node.js)
console.log(__dirname); // "/pfad/zum/verzeichnis"
console.log(__filename); // "/pfad/zum/verzeichnis/datei.js"

// ESM -- diese Variablen gibt es nicht!
// Stattdessen:
import.meta.url; // "file:///pfad/zum/verzeichnis/datei.js"
```

4.5 `<script>` vs. `<script type="module">` – Alle Unterschiede

Eigenschaft	<code><script></code>	<code><script type="module"></code>
Scope	Global (window)	Modul-Scope (isoliert)
Strict Mode	Optional	Immer aktiv
Ausführung	Sofort beim Parsen	Deferred (nach DOM bereit)
Mehrfaches Laden	Wird mehrfach ausgeführt	Wird nur einmal ausgeführt
Import/Export	Nicht verfügbar	Verfügbar
Top-level await	Nicht möglich	Möglich
CORS	Gleich-Origin only	CORS-Modus (strenger)

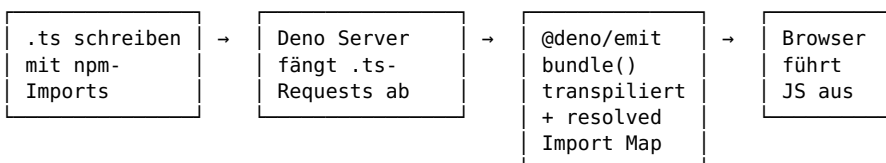
4.6 Import Assertions – Neue Syntax für spezielle Importe

```
// JSON-Dateien importieren -- mit Import Assertion
import personen from "./personen.json" with { type: "json" };
```

Die `with { type: "json" }` Syntax ist ein **Import Assertion** – sie teilt dem Modulsystem mit, dass die Datei als JSON behandelt werden soll. Ohne diesen Hinweis würde der Parser versuchen, die JSON-Datei als JavaScript zu parsen, was fehlschlägt.

5. Zusammenfassung und Best Practices

5.1 Die TypeScript-zu-Browser-Pipeline



5.2 Best Practices

1. **import/export verwenden** – niemals `require()` in Browser-Code
2. `<script type="module">` oder `<script ... defer>` verwenden
3. **addEventListener** statt **onclick** im HTML – Module haben isolierten Scope
4. **Import Map in deno.json** pflegen – zentrale Stelle für alle Modul-Auflösungen
5. **ESM.sh** für npm-Pakete im Browser nutzen – `?target=es2022` nicht vergessen
6. **Typen explizit angeben** bei Funktionsparametern und -rückgaben; lokale Variablen inferieren lassen
7. **TypeScript direkt im HTML referenzieren** (`<script src="app.ts">`) – der Transpile-Server erledigt den Rest

5.3 Starten des Transpile-Servers

```
# Deno installieren (einmalig)
curl -fsSL https://deno.land/install.sh | sh

# Transpile-Server starten
cd deno_transpile
deno task dev

# Server läuft auf http://localhost:8000
# TypeScript-Dateien werden automatisch bei Änderungen neu transpiliert
```